

Timing Experiments in Python and Jupyter Notebooks

In a **Jupyter Notebook**, compare the runtimes of various search algorithms when searching for a randomly selected integer within a list of integers. Ultimately, you should compare runtimes of various search algorithms (Linear *vs.* Binary *vs.* *an-alternate-search-of-your-choice*) when searching in lists, L , of various sizes:

$\text{len}(L) = \{\text{Kilo}=10^3, \text{Mega}=10^6, \text{Giga}=10^9, \text{Tera}=10^{12}, \text{and Peta}=10^{15}\}$

on two different computers (your laptop, one of csLab's Linux boxes).

Take care to only time the search functions. Of course, some algorithms (e.g., binary search) assume that the list of values have been previously sorted. Don't include the time to sort since we are comparing the execution times of **search algorithms**. Of course, you must make sure that you are searching for "random" values that appear in the lists; i.e., you should not always search for a 3 (near the front of the list), but rather each time, search for a random assigned value.

For measuring runtimes, use the Python **timeit** module, rather than "magic commands" in Jupyter Notebooks.

In a separate file, unit test each of your search functions using **pytest**.

All print format statements use f-strings. All functions should use one-line docstrings. Use type hinting for all the arguments and return types in all functions. Before submission, run your code through a "linter", e.g., mypy <https://www.pythontutorial.net/python-basics/python-type-hints/>

Compute and print the descriptive stats (min, avg, max) runtime for each search (linear vs. binary) for various List (L) sizes. (Please be careful: you *really* must know the difference between **timeit**'s arguments of number and repeat).

GRADE KEY - Assignment #1 Timing Functions

Average Effort

Assumes a Jupyter Notebook: **timing.ipynb** file ☐ YES ☐ No

All/Some/no functions unit tested (in a separate file that uses **pytest**).

All/Some/no functions include:

☐ good PEP8 style ☐ use of **camelCase** names ☐ **type hints**
☐ **docstrings** ☐ extra documentation within each function (e.g., what are the arguments?), **inline comments**

Implements Linear *and* Binary Search and produces a report comparing their runtimes when searching through lists (L) of various sizes, $\text{len}(L) = \{10^3, 10^6, 10^9, 10^{12}, 10^{15}\}$ on two different computers.

OUTPUT(s)

Succinct summary of **results printed to standard output (stdout)**. In our case, the output will appear in your Notebook. Note: this output need not be "fancy" (that will be in your report) but it should allow the user to quickly see the results.

Trial results output to a .csv file:

Above Average Effort

You do all the work above. Starting with your .csv output, prepare a **separate report (.pdf)** that (a) describes the experiment and (b) summarizes the results in a neat, professional-looking set of Tables and Figures (graphs). You can do the graphing in Excel and/or Sheets, taking care to use **log**-axes.

The report, Tables, and Figures must define the units used (e.g., all times are in microseconds). Of course, all Tables and Figures have a unique number and caption that explains what the reader is viewing. Include specs on each computer used.

Superior Effort

You do all the work above and ...

☐ Implement and compare a **third search algorithm** (e.g., Interpolation, Exponential, Fibonacci Search)

☐ Add **matplotlib** (or **plotly**) **graphs** of the runtimes, taking care to use log-axes. Graphs can be produced in your Notebook, but should also be copied to and included in your report. Your chatBot will be your friend to help you find examples of plotting values in Python. **Please cite/reference any code that you use from others or bots.**